第一次作业：watertank模糊模型设计报告

在本次作业中，我设计了一个自定义的模糊控制类 `Policy.m`，用于精确跟踪和控制非线性一阶水箱的目标水位。在经过多次调试后，我发现水箱在“大排量满负荷爬升（如目标为25）”和“低水位重力阶跃放水（如目标为10）”这两种情况下的物理响应和对阻尼的需求是截然不同的。因此，我的最终控制策略结合了**前馈补偿计算**与基于目标值切换的**增益调度（Gain Scheduling）双模模糊控制。**

一、控制算法原理

我的控制器的总输出由两部分物理量叠加组成：

$$U_{total} = U_{fuzzy} + U_{feedforward}$$

1. 动态前馈平衡部分（`U_feedforward`）

为了消除稳态下的系统误差，我加入了前馈控制逻辑。根据基本的流体静力学原论（伯努利原理），水箱底部的泄露排速与当前水位高度的平方根成正比。因此，维持固定目标水位所需的进水基准量为：

$$U_{feedforward} = Gain \times \sqrt{\max(sp, 0)}$$

在代码实现中，`Gain` 的大小是依靠读取配置文件 `sys.ini` 中的进出水系数比例（`op / ip`）动态求得的。这相当于直接向系统提供了一个理论平衡点。

2. 双策略模糊控制部分（`U_fuzzy`）

在有了前馈做粗调基准后，我需要依赖模糊系统处理跟踪中的动态误差。为了脱离对 MATLAB 官方 `mamfis` 等外部第三方工具箱的调用依赖，也为了深入理解模糊逻辑，我在代码内部手写了一个轻量的模糊映射解算引擎 `fuzzify7`。

我将观测到的误差转化为 7 个模糊集合：NB（负大），NM（负中），NS（负小），ZE（零），PS（正小），PM（正中），PB（正大）。考虑到前文提到的两种特殊水箱工况，我设计了两套并行的模糊推断阵列，并按照目前目标指令（sp）自动切换。

情况 A：高水位爬升阶段（针对 `sp > 15`）

这是为了应对水箱缓慢注水时的过冲现象，我采用了一套 **二维模糊 PD 控制器。**在代码结构中，这对应了 `sp > 15` 的增益调度逻辑：

```
if sp > 15
    % 工况 A (高水位满负荷爬升): 采用二维模糊 PD 矩阵
    e_range = 25; de_range = 2.0; du_range = 50;

    e_n  = max(-6, min(6, e  / e_range  * 6));
    de_n = max(-6, min(6, de / de_range * 6));

    e_mf  = Policy.fuzzify7(e_n);
    de_mf = Policy.fuzzify7(de_n);

    % 定义的 7x7 二维决策推断控制矩阵 R
    R = [
         1,  1,  1,  1,  1,  2,  2;
         1,  1,  2,  2,  2,  3,  3;
```

```
        2,  2,  3,  3,  3,  4,  4;
        3,  3,  3,  4,  5,  5,  5;
        4,  4,  5,  5,  5,  6,  6;
        5,  5,  6,  6,  6,  7,  7;
        6,  6,  7,  7,  7,  7,  7;
    ];
```

- **输入**：选用双状态，即水位误差 `e = sp - H`，以及它随时间的变化率 `de = e - last_e`。这里的误差变化率起到了“微分阻尼”的作用，从而帮助阀门在前瞻趋势上提前动作。
- **规则矩阵**：依据上述的经验矩阵 `R`，系统利用 `w = min(e_mf, de_mf)` 提取对应交集论的最小激活度，并据此决定输出响应等级。由于考虑了速度 `de`，当系统检测到误差不大但上升过快时，能及时输出截断动作进行有效的“刹车”。

情况 B：低水位极速下落（针对 `sp <= 15`）

这是针对实验后半段水箱目标突然归 10 的特殊设计。由于卸水纯看重力漏孔，没有主动排水阀门，本来就非常慢；如果在此时继续采用带有微分项的矩阵，系统会因为检测到超高的下落速度，反而把进水阀打开试图减速防抖，使得放水极其拖沓。

- **输入与映射**：我果断去除了微分项，改用 **一维高敏度纯 P 模糊映射。**
- **工作机制**：将误差论域压缩至仅 `[-1.2, 1.2]`。换言之，只要目标水位还在脚底下，就立刻驱动阀门进行 100% 满偏（也就是 0 阀门开度）进行死区切断；一旦回到极为苛刻的目标容差内，再用重心法推算出精确的一维比例输出来软着陆。这一部分的工程代码为了追求极速和敏锐度，结构极其紧凑精简：

```
else
    % 工况 B (低水位阶跃下降): 采用一维标准自编模糊推断
    % 针对低水位的极速放水需求，极度缩小物理感知域限幅
    e_range_2 = 1.2;
    du_range_2 = 30.0;

    e_n = max(-6, min(6, e / e_range_2 * 6));
    e_mf = Policy.fuzzify7(e_n);

    % 一维线性对应：提取基础论域中心点直接按重心法比例加权提取
    centers = [-6, -4, -2, 0, 2, 4, 6];
```

二、代码实现细节与安全兜底

在 `Policy.m` 的代码编写中，除了数学核心，我也加入了一些对现实数字系统安全性的考量设计。

目标突变防冲击机制

由于我在前半段中运用了微分项 `de`，当控制目标 `sp` 由 25 毫秒级突变切换回 10 时，数学上会瞬间产生一个无法理喻的巨大导数（伪导数值），这种震荡如果在工业上会令执行器直接击穿损毁。
考虑及此，我在入口处手动编写了一个防冲击的过滤锁：

```
if obj.last_sp > 0 && abs(sp - obj.last_sp) > 1
    obj.last_e = sp - H; % 目标一旦跳变，立刻重对齐历史基准以平息微分振荡
end
```

动作去模糊化

对于提取出的任何模糊动作特征，我都一律使用经典、泛化能力最强的“加权平均重心法（`centroid`）”进行求解：

$$du = \frac{\sum(du_{mf} \times centers)}{\sum(du_{mf})} \times scale_{out}$$

在 MATLAB 核心中我具体编写的面相物理求解代码如下。由于考虑了边界触发异常，我加入了微小安全容差作为分母保护：

```
sw = sum(du_mf);
if sw < 1e-10
    du_n = 0; % 平息 0 触发奇点
else
    du_n = sum(du_mf .* centers) / sw;
end

du = du_n / 6 * du_range; % 去模糊化比例还原缩放放大，回归物理域限

% 系统依据伯努利原理提取抵消目前低高度泄漏等效的前馈稳定量
u_feedforward = obj.ff_gain * sqrt(max(sp, 0));
% 进行稳态融合后使用底线截断防止水流倒吸
u_total = du + u_feedforward;
act_val = [max(0, u_total)];
```

三、实验总结体会

通过独立完成对水箱仿真控制的第一次任务包，我的专业技能得到了很大的提升。我完全自主、不依赖 MATLAB 官方库死板包敲出了属于自己的底层模糊逻辑推断引擎 `fuzzify7`（利用原生代码执行轻量高效的边界截断和均匀三角形映射）：

```
% Static method 局部节选：纯手写的高效分类隶属度解算探针
function mf = fuzzify7(x)
    mf = zeros(1, 7);
    c  = [-6, -4, -2, 0, 2, 4, 6];
    for i = 1:7
        if i == 1
            if x <= c(1),         mf(1) = 1;
            elseif x < c(1) + 2, mf(1) = (c(1) + 2 - x) / 2;
            end
        ... % 相似推断略
end
```

这种知其然更知其所以然的代码实践，也让我深切体会到了在课堂上学到的交集、边界隶属函数分配、重心求解法等数学理论究竟是如何具像化为一行行灵动运作的代码的。这无论在实践还是理论认知上，都令我收获颇丰！

四、结果展示

phase1:

```
Current Running Time: 3.13 s, Current Score: 9866.08
Current Running Time: 3.14 s, Current Score: 9866.08
Current Running Time: 3.15 s, Current Score: 9866.08
Current Running Time: 3.16 s, Current Score: 9866.08
Current Running Time: 3.17 s, Current Score: 9866.08
Current Running Time: 3.18 s, Current Score: 9866.08
```

phase2:

```
Current Running Time: 9.4 s, Current Score: 9753.2882
Current Running Time: 9.41 s, Current Score: 9753.2882
Current Running Time: 9.42 s, Current Score: 9753.2882
Current Running Time: 9.43 s, Current Score: 9753.2882
Current Running Time: 9.44 s, Current Score: 9753.2882
Current Running Time: 9.45 s, Current Score: 9753.2882
Current Running Time: 9.46 s, Current Score: 9753.2882
Current Running Time: 9.47 s, Current Score: 9753.2882
Current Running Time: 9.48 s, Current Score: 9753.2882
Current Running Time: 9.49 s, Current Score: 9753.2882
Current Running Time: 9.5 s, Current Score: 9753.2882
Current Running Time: 9.51 s, Current Score: 9753.2882
Current Running Time: 9.52 s, Current Score: 9753.2882
```